AgriPulse — Engineering Progress Report

AI Predictive Maintenance for Irrigation Pumps — Graduation Project, AI Engineering Program, Faculty of Engineering, Mansoura University

---

1. Project Overview

AgriPulse is a predictive-maintenance AI platform for centrifugal irrigation pumps: it learns normal pump behaviour from multi-sensor telemetry, detects early deviations, converts them into a calibrated failure-risk score with a 6-to-12-hour warning horizon, and generates a standards-grounded maintenance work order through a Retrieval-Augmented Generation (RAG) engine. The full scope and 67-task execution plan are documented in *docs/*; this report covers progress on the first two workstreams: WS1 (data pipeline) and WS2 (model training & evaluation).

2. Work Completed — WS1 (Data Pipeline)

Tasks T08–T13 of the plan: implemented and run end to end against the real 220,320-row sensor dataset — not just designed, but working, tested code producing real output.

Task	Deliverable	Status
T08 — Ingest & profile dataset	profile_report.json	Done — verified
T09 — Clean data & sensor handling	sensor_handling.md	Done — policy corrected (see §4)
T10 — Labeling (WARNING / RUL targets)	labeling.py, 3 horizons (6h/8h/12h)	Done
T11 — Leave-one-failure-out validation	validation.py, 7 folds	Done
T12 — Feature engineering library	features.py (rolling/EWM/FFT/cross-sensor)	Done
T13 — Versioned feature table	features_v1.parquet, 1,053 columns	Done

Repository is organized by workstream (ml/, rag/, backend/, frontend/, iot/) and under git version control.

3. Work Completed — WS2 (Model Training & Evaluation)

Tasks T14–T20: every model is trained and evaluated leave-one-failure-out (7 folds — one real failure event held out per fold, never seen in that fold's training data). All numbers below are real, reproducible results, not placeholders or projections.

Task	Headline result	Status
T14 — Anomaly baselines (IsoForest/PCA/Mahalanobis)	1–2 / 7 events detected	Done
T15 — LSTM-Autoencoder	1 / 7 detected (first pass, 4 epochs)	Done
T16 — Supervised risk (XGBoost/LightGBM)	3 / 7 detected — see bug note (§4)	Done
T17 — Calibration + Green/Yellow/Orange/Red levels	isotonic calibration	Done

Task	Headline result	Status
T18 — Persistence/hysteresis alert logic	self-tested, no false flicker	Done
T19 — RUL regressor	MAE ~194 min on a 720-min horizon	Done
T20 — Model registry & model card	ml/models/v1/model_card.json	Done

Detection rates look modest (best model: 3/7) — this is honest, not a weak result: with only 7 failure events total, a leave-one-failure-out fold tests genuine generalization to an event the model never saw, which is a much harder and more meaningful bar than in-sample accuracy.

4. Key Findings During Engineering

Finding 1 — two sensors had multi-day data gaps directly overlapping two failure events

The original plan called for forward-fill / linear interpolation on sensor_50 and sensor_51. Profiling the actual data showed this would have been a mistake: sensor_50 has a single ~77,000-minute (53-day) gap that swallows breakdown event #7 entirely, and sensor_51 has a ~15,356-minute (10.6-day) gap covering the full pre-failure warning window of event #5 — the largest failure in the dataset (140.5 hours of downtime).

Filling gaps that size would have fabricated weeks of fake sensor readings through the exact events the model is supposed to learn from. The pipeline now leaves long gaps as missing and adds a `_was_missing` indicator column instead, and only interpolates genuinely short gaps (recalibrated threshold: 30 minutes, based on the actual gap-length distribution in the data). Documented in ml/reports/sensor_handling.md.

Finding 2 — a calibration bug made the first model results falsely look perfect

The first T16/T17 run reported a suspicious "perfect" result: every one of the 7 held-out failures detected, every single one at the very first minute of its 12-hour warning window. That pattern — not gradual, not varying per event — was the tell that something was wrong rather than something to celebrate.

Root cause: isotonic probability calibration collapsed the near-perfectly-separated NORMAL class to a calibrated probability of exactly 0.0, which made the 99th-percentile alert threshold also exactly 0.0 — and since a probability can never be negative, literally every row in the dataset trivially satisfied "score >= 0". Fixed by computing the alert threshold from the raw, uncalibrated probability instead. The honest, real result after the fix is 3/7 (§3). See ml/src/supervised_risk.py for the full writeup.

5. Validated Dataset Facts

All headline numbers quoted in the planning documents were checked against the real CSV and confirmed accurate:

Claim	Verified value
Total rows	220,320 (Apr 1 – Aug 31, 2018, 1-minute intervals, no gaps/duplicates)
Sensor channels	52 (sensor_00 .. sensor_51)
Target distribution	NORMAL 93.43% / RECOVERING 6.57% / BROKEN 0.003% (7 rows)

Claim	Verified value
Breakdown events	7, timestamps match exactly to the minute
sensor_15	100% missing — dropped

6. Open Items Needing Supervisor / Team Input

- **Dataset provenance:** the data's structure (perfectly regular, zero gaps/duplicates, anonymized sensor_00..51 naming) matches a known public predictive-maintenance pump dataset rather than original Egyptian field telemetry. Should be cited properly in the thesis rather than implied as proprietary field data.
- **Team & supervisor fields:** the official university template still has placeholder names, credit hours, and GPA — needs to be filled before submission.
- **External accounts:** LLM API key (OpenAI/Gemini), a cloud database (TimescaleDB/Neon), and a Vercel account are needed once work reaches the RAG engine and backend/deployment stages (WS4, WS6, WS9) — not required yet.
- **Budget:** the ~$33 allocated for cloud/LLM tokens looks tight if development involves many iterations plus a live demo.

7. Next Steps

WS3 (T21–T25): SHAP-based explainability on the risk model, per-sensor reconstruction error from the autoencoder, and formal event-level evaluation reporting with ablations. Then WS4 (T26–T33): build the knowledge base (ISO 10816-3, FMEA, O&M manuals) and the RAG engine — this is where the LLM API key noted in §6 becomes necessary.

Generated from the project repository's committed code and reports — every figure above is reproducible by re-running the referenced scripts.